

1 Sisteme de control al versiunilor

1.1 Introducere. Ce și de ce

Un sistem de control al versiunilor (*engl.* version control system - VCS) înregistrează modificările făcute asupra unui produs software într-o bază de date specială, numită repository, (din *engl.* pentru depozit, prescurtat uneori *repo*). Se **înregistrează modificările asupra codului** împreună cu **cine, când și de ce** a făcut respectivele modificări. În anumite situații, dacă se dorește, **se poate reveni asupra modificărilor**, adică acestea **sunt reversibile**. De asemenea, un VCS **facilitează cooperarea** pe același proiect, facilitând gestionarea contribuțiilor de la mai mulți participanți.

Fără un sistem de control al versiunilor, proiectele se gestionează, în general, în foldere cu diferite denumiri, de exemplu utilizând data ultimei modificări în denumire sau utilizând nume de versiuni. Există dificultăți în a gestiona astfel diferitele versiuni, chiar și când este o singură persoană care se ocupă cu partea de versionare. Este o risipă de spațiu, de timp, mai ales în cazul proiectelor mari.

Tabelul 1. Exemple de denumiri ale surselor pentru diferite versiuni

 v1.0	 2024_03_19
 v1.1	 2024_03_26
 V1.2	 2024_04_01

Atunci când lucrează mai multe persoane pe o aceeași versiune, lucrurile devin și mai dificile. Modificările trebuie discutate printr-un canal de comunicare (email, mesagerie de orice tip) și cineva trebuie să gestioneze manual versiunea rezultată prin combinarea diferitelor versiuni de cod. Pot apărea probleme în care un membru al echipei lucrează pe o versiune neactualizată de cod.

Tabelul 2. Combinarea a diferite versiuni la care lucrează simultan mai mulți membri ai echipei

Versiuni programator 1		Versiuni programator 2
 v1.0		 v1.0
 v1.1		 v1.1
 V1.2		
		

1.2 Exemple de sisteme de control al versiunilor

Cel mai folosit sistem de control al versiunilor este **Git**, creat de către **Linus Torvalds**, cel care a scris primul **nucleu Linux** și care a fost mult timp, cel care a gestionat creșterea acestui cod. Nucleul de Linux, pe care se bazează atâtea sisteme de operare, a pornit de la câteva mii de linii de cod, scrise de Linus Torvalds, ajungând să aibă milioane de linii de cod, împărțite pe diferite module și subproiecte, la care contribuie oameni din întreaga lume, fiind un proiect de tip **open source**. Din dificultatea gestionării contribuțiilor la un proiect de asemenea dimensiuni, a reieșit necesitatea acestui tip de instrument.

Există numeroase sisteme de control al versiunilor, care se împart în două categorii:

- Centralizate:
 - o Proiectul este salvat pe un server, la care fiecare membru al echipei se conectează.
 - o Problemă: single point of failure – dacă sunt probleme cu serverul, nimeni nu poate lucra
 - o Exemple:
 - Apache Subversion (open source)
 - Microsoft Team Foundation Server
- Distribuite:
 - o Fiecare contributor are o copie a proiectului și a istoricului de modificări pe propria mașină și
 - o Exemple:
 - Git
 - Mercurial

În acest curs, ne vom axa pe prezentarea Git, fiind cel mai folosit sistem de versionare (conform unui sondaj realizat de Stackoverflow, 93% din programatori folosesc Git). Conceptele de bază sunt aceleași în toate sistemele de tip VCS. Git poate fi folosit gratuit, este de tip open source, este rapid și scalează ușor și pentru repositories de mari dimensiuni.

1.3 Variante de utilizare git

Git se poate folosi prin:

- Interfața de tip linie comandă (*engl.* Command Line Interface – CLI)
- Interfețe de tip GUI
- Add-in/ plugin instalat în diferite editoare de cod și medii de dezvoltare (VSCode, Eclipse, IntelliJ, etc.)

Există numeroase interfețe de tip GUI pentru Git. O listă a acestora este disponibilă la adresa oficială a git, git-scm.com.

GUI Clients

Git comes with built-in GUI tools for committing ([git-gui](#)) and browsing ([gitk](#)), but there are several third-party tools for users looking for platform-specific experience.

If you want to add another GUI tool to this list, just [follow the instructions](#).



26 Linux GUIs are shown below ↓

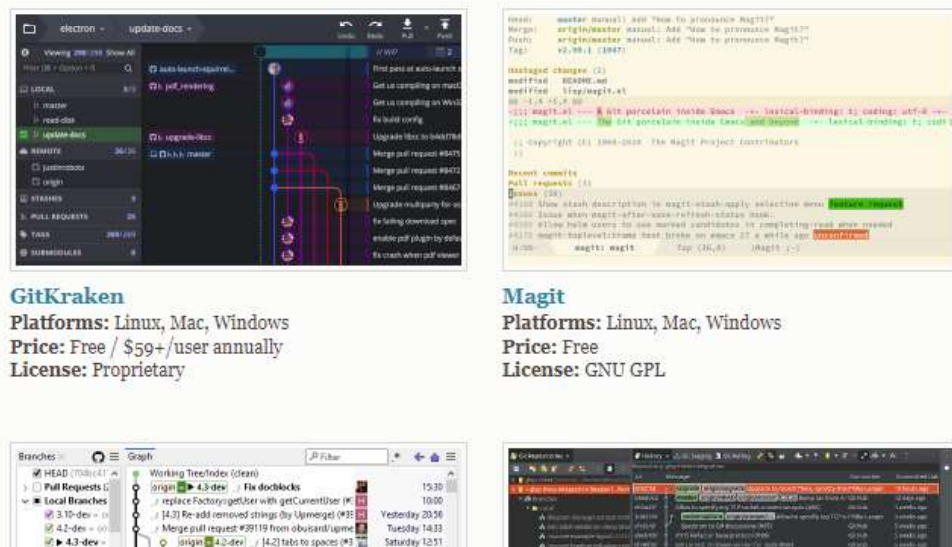


Figura 1. Clienți GUI pentru git grupați pe diferite platforme

Interfețele de tip GUI implementează cele mai utilizate comenzi și, în general, prezintă funcționalitățile necesare. Pot exista situații în care acestea nu implementează comenzile necesare de git, caz în care trebuie revenit la interfața de tip CLI. Printre cele mai populare opțiuni de GUI se numără GitCracken și SourceTree.

1.4 Concepte de bază în git

repository / repo – se referă la proiectul ale cărei versiuni vor fi controlate prin git, compus din toate fișierele de diferite tipuri create pentru el. Tehnic, în el trebuie să existe un folder ascuns (cu denumirea `.git` – observăm, `.` în denumire – denotă faptul că este ascuns) care se află în directorul principal al proiectului

working directory – directorul de lucru: în principiu locația proiectului pe calculatorul de lucru

branch – o versiune separată a repository-ului principal. Permit lucrul simultan la diferite părți ale proiectului.

main – este branch-ul principal al proiectului. Se creează automat atunci când se creează un repository. Într-un repository mai nou (după oct. 2020) se numește main.

master – este denumirea veche pentru branch-ul main. Într-un repository mai vechi (înainte de oct. 2020) se numește master.

staging area – este componenta care ține evidența a ceea ce este modificat în working directory și se dorește a fi actualizat (sincronizat) în repository

head – este o referință la versiunea curentă de repo

index - o altă denumire pentru staging area

commit – acțiune care se referă la salvarea modificărilor din staging area sau direct din working directory în repo (în head)

snapshot – o instanță a proiectului la un moment dat. Se gestionează prin folder-ul .git din directorul principal al proiectului. Un snapshot se realizează cu acțiunea de commit – comanda git commit

O vizualizare bună a conceptului de branch este oferită în Figura 2

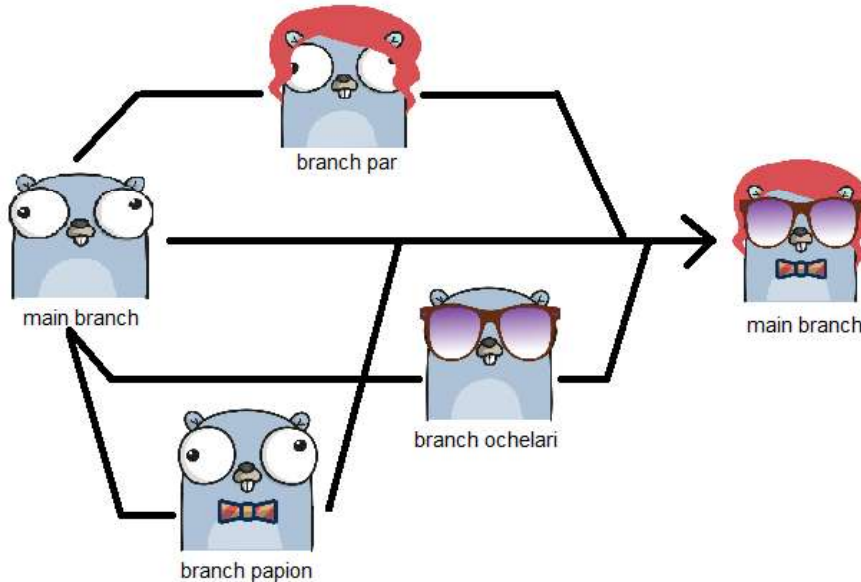


Figura 2. Reprezentare concept de branch în contextul git

1.5 Instalare git

Pentru a verifica dacă este deja instalat git, se deschide un terminal/ o fereastră Command Prompt:

- Linux: Ctrl+ Alt+T sau tastă windows + scris *term*
- Windows: tastă windows + scris *cmd*
- MacOS: taste Cmd + Space + scris *term*

Scriem comanda git -- version și verificăm ieșirea comenzii.

```
C:\Users\ica>git --version
git version 2.29.2.windows.1
```

```
$ git --version
bash: git: command not found
```

```
ik@IKPC:~$ git --version
git version 2.25.1
```

Figura 3. Exemple de ieșiri ale comenzii git --version

Vom afla ori versiunea de git instalată ori faptul că nu a fost găsită comanda, ceea ce înseamnă că git nu este instalat (uneori poate însemna că nu este găsit în locațiile în care se caută comenzile, dar această situație este mai greu de obținut, ținând cont de faptul că instalarea git se face în mod implicit în locațiile predefinite).

Dacă git nu este instalat, se instalează așa cum se instalează un program pentru sistemul de operare folosit. Ghiduri de instalare se găsesc pe pagina oficială a git la adresa www.git-scm.com/downloads.

1.6 GitHub

GitHub este o platformă care implementează git pentru controlul versiunilor. Deși a pornit ca un serviciu pentru acest lucru, oferă funcționalități foarte variate, printre care și:

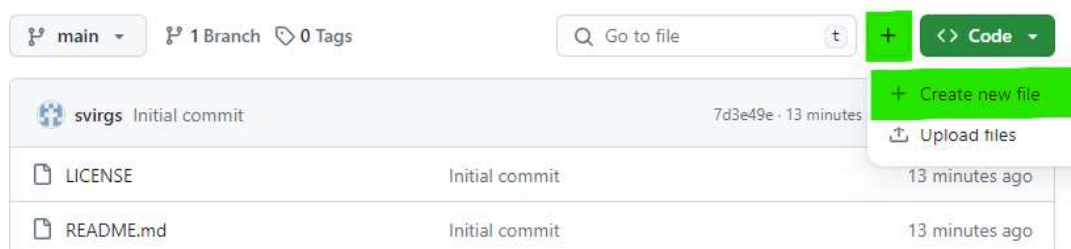
- Realizarea de documentații de tip wiki
- Gestionarea de proiecte prin gestiunea task-urilor care sunt de făcut, făcute sau în progres.

1.7 Exerciții

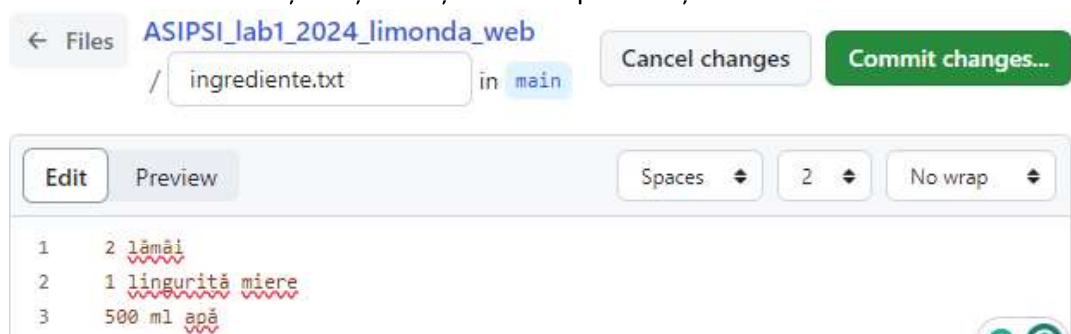
1. Dacă nu aveți, creați un cont pe platforma GitHub
2. Creați un nou repository din interfața web:
 - a. cu numele ASIPSI-Lab1-Limonada-web;
 - b. în descriere puteți pune un text explicativ
 - c. de tip Public – oricine poate vizualiza acest repo
 - d. adăugați un fișier README în care scrieți un text care să explice care este scopul laboratorului.
 - e. Adăugați un fișier de tip .gitignore.
 - f. Alegeți o licență open source de tip GNU GPLv3.
3. Navigați prin interfața web. Observați că aveți două fișiere deja create: licența și fișierul README.md.
4. Din interfața web creați două fișiere: Ingrediente.txt și Instrucțiuni.txt cu conținutul descris mai jos.

Ingrediente.txt	Instrucțiuni.txt
2 lămâi 1 linguriță miere 500 ml apă	Spălați lămâile Storceți lămâile Adăugați mierea Adăugați apa

- a. Adăugați un fișier:



- b. Scrieți conținutul și un nume pentru fișier



- c. Salvați fișierul apăsând pe butonul „Commit changes” și completând formularul

Commit changes

Commit message

Create ingrediente.txt

Extended description

am adaugat lista de ingrediente

☒ Commit directly to the main branch

☐ Create a new branch for this commit and start a pull request

[Learn more about pull requests](#)

Cancel

Commit changes

d. Repetați pașii pentru fișierul Indicații.txt

5. Din interfața web, modificați lista de ingrediente adăugând o linguriță sare de lămâie în fișierul ingrediente.txt.

CodeBlame

RawCopyDownloadEdit

```
1 2 lămâi
2 1 linguriță miere
3 500 ml apă
```

- Apăsați pe creionul din imagine, editați fișierul
- Apăsați pe butonul „Commit changes” și completați formularul.

Vom repeta aceleași lucruri din interfața de tip CLI

6. Instalați git/ verificați versiunea instalată. Dacă nu este instalat, descărcați și instalați versiunea corespunzătoare sistemului de operare folosit de la www.scm-git.com
7. Deschideți un terminal/ aplicația git bash
8. Configurați git cu setările inițiale astfel:
 - a. `git config --global user.name "numele de utilizator de git"`
 - b. `git config --global user.email "adresa email cont git"`
 - c. `git config --global core.editor "nume editor folosit"`
 - i. notepad în Windows sau vim în sisteme bazate pe Linux sau VSCode. Pentru VSCode, se completează în loc de „nume editor folosit” „code –wait”
 - d. `git config --global init.defaultBranch main`

Abia acum începem cu cerințele realizate în interfața web.

9. Verificați că au fost salvate configurările în fișierul de configurare cu comanda `git config --list`
10. Creați un director cu numele retetaLimonada: `mkdir retetaLimonada`
11. Verificați conținutul directorului cu comanda `ls` sau `dir` și apoi cu `ls -a` sau `dir /a`. (`ls` în sisteme bazate pe Linux, `dir` în Windows).
12. Faceți ca acest director să fie repository git, cu comanda:

```
git init -b main
```

13. Verificați conținutul directorului cu comanda `ls` sau `dir` și apoi cu `ls -a` sau `dir /a`. (`ls` în sisteme bazate pe Linux, `dir` în Windows)
14. Creați cele două fișiere: `Ingrediente.txt` și `Instrucțiuni.txt` cu conținutul descris mai jos.

Ingrediente.txt	Instrucțiuni.txt
2 lămâi 1 linguriță miere 500 ml apă	Spălați lămâile Stoarceți lămâile Adăugați mierea Adăugați apa

- a. Folosind vim *nume_fisier*
- b. sau notepad *nume_fisier*
- c. sau code *nume_fisier*

Nota: în vim, pentru a insera text se apasă mai întâi tasta i (se intră în modul Insert și pentru a salva modificările, se apasă pe Esc și apoi se tasează :wq)

15. Verificați ce fișiere se află în staging area utilizând comanda `git status` și comanda `git status -s`.
16. Adăugați cele două fișiere utilizând comanda `git add`

```
git add ingrediente.txt
git add Instrucțiuni.txt
```

 sau

```
git add ingrediente.txt Instrucțiuni.txt
```

 sau

```
git add .
```
17. Verificați ce fișiere se află în staging area utilizând comanda `git status` și comanda `git status -s`
18. Faceți o modificare în fișierul `ingrediente.txt` adăugând 100 ml apă minerală.

19. Verificați ce fișiere se află în staging area utilizând comanda `git status` și comanda `git status -s`.
20. Creați un snapshot al modificărilor folosind comanda `git commit` cu opțiunea `-m` pentru a adăuga un mesaj scurt care descrie scopul commit-ului. Pentru prima urcare, se poate folosi un mesaj similar cu initial commit.
21. Urcați cele două fișiere în repository online.

Pentru a ne autentifica pe GitHub din line de comandă, este necesară configurarea unui token de acces pentru GitHub, astfel:

1. Generați un token

Accesați GitHub, intrați în setări → Developer Settings → Personal access tokens → Tokens (classic)

Completați formularul de generare a unui nou token:

- Completați câmpul Note
- Completați câmpul pentru expirare: setați pe Never
- Selectați câmpurile de acces pentru acest token: bifați toate căsuțele
- Apăsați butonul de generare
- Salvați token-ul generat (copiați-l și notați-l)

2. Configurați token-ul în să fie folosit în sistemul de operare

- a. Pe windows: Deschideți Credential Manger → Windows Credentials → Add a generic credential și completați formularul cu adresa `git:https://github.com`, User name: numele de utilizator github, Password: tokenul generat anterior
- b. Pe sisteme Linux: când vi se cere parola, furnizați token-ul și apoi folosiți comanda: `git config --global credential.helper cache`

- a. Creați un repository gol (fără licență, fără Readme și fără .gitignore) în GitHub cu numele `retetaLimonada`. Copiați linkul sau și citiți un pic despre pașii de început expuși și în screenshot-ul următor.

Quick setup — if you've done this kind of thing before

 Set up in Desktop or  HTTPS  SSH `https://github.com/svirs/retetaLimonada.git` 

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# retetaLimonada" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/svirs/retetaLimonada.git
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin https://github.com/svirs/retetaLimonada.git
git branch -M main
git push -u origin main
```

...or import code from another repository

You can initialize this repository with code from a Subversion, Mercurial, or TFS project.



- b. Faceți ca acest repo să fie sincronizat cu cel creat pe GitHub (să fie sursa de conținut pentru cel creat online) cu comanda:

```
git remote add origin https://github.com/svirgs/retetaLimonada.git
```

(înlocuiți adresa repo-ului cu adresa repo-ului creat mai devreme în GitHub)

- c. Folosiți comanda `git push -u origin main`.